Parul® University
Vadodara, Gujarat

NAAC GRADE A++

LAB MANUAL – SOCIAL MEDIA ANALYTICS & SENTIMENT ANALYSIS

Study Guide

Mohd Affan Sheikh
Assistant Professor
MCA, PIET.
Parul University

INDEX

SR. NO.	Date	Practical Title	Page no.
1		Study of Webpage Structure Using Source Code	
2		SEO Analysis of a Website (H1, Meta Tags, Keywords) Using Google Search Console	
3		Data Collection Using Twitter API	
4		Data Collection Using Web Scraping (BeautifulSoup)	
5		Data Preprocessing Using Pandas – Cleaning and Standardization	
6		Data Preprocessing Using Pandas – Missing Values, Duplicates and Data Types	
7		Tokenization and Stop Word Removal	
8		Stemming and Lemmatization	
9		Implement Bag of Words Model	
10		Implement TF-IDF	
11		Sentiment Analysis Using Text Blob	
12		Sentiment Analysis Using NLTK	
13		Build ML Model (Naïve Bayes / Logistic Regression)	
14		Data Visualization Using Matplotlib / Seaborn	
15		Mini Project: Social Media Sentiment Analysis (End-to-End)	

Practical No. 1
Study of webpage structure using source code.

AIM: To study the structure of a webpage by inspecting its HTML source code and identifying common HTML elements, attributes, headings, links, images, metadata and page sections.

Theory: A webpage is primarily built using HTML (Hyper Text Markup Language). The browser reads HTML tags and builds the Document Object Model (DOM). The source code shows how content is organized.

Program:
File name: Index.HTML

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Learning Portal</title>
   <meta name="description" content="Learning resources for students">
</head>
<body>
   <header>
      <h1>Student Learning Portal</h1>
      <p>Welcome to the learning page.</p>
   </header>


   <section>
      <h2>Web Development</h2>
      <a href="https://example.com">Visit Resource</a>
      <img src="logo.png" alt="College Logo">
   </section>
</body>
</html>
```

Output:

The browser displays the webpage with the heading “Student Learning Portal”. The source code contains the title, meta description, H1, H2, link and image elements.

Practical No. 2
SEO analysis of a website (H1, meta tags, keywords) using Google Search Console.

AIM: To analyze basic SEO information of a website using Google Search Console, including indexing status, search queries, pages, impressions, clicks and page-level SEO elements.

Theory: To analyze basic SEO information of a website using Google Search Console, including indexing status, search queries, pages, impressions, clicks and page-level SEO elements.

Program:
No programming code is required for this practical. The analysis is performed using Google Search Console and browser source/Developer Tools.

Example observation: Clicks = 125, Impressions = 4,500, CTR = 2.8%, Average position = 18.6. Page source may show one <h1>, a <title>, and a meta description. Values depend on the selected verified property and date range.

RESULT: Basic SEO and Google Search performance elements were analyzed using Search Console and webpage source code.

Practical No. 3
Data collection using Twitter API.

AIM: To collect recent public posts from Twitter/X using its API and store selected fields for social media analysis.

Theory: An API (Application Programming Interface) allows a program to request data from a service in a structured way. The Twitter/X API can provide public post data when the required developer access and permissions are available.

Program:
File name: Twitter.py

```
import tweepy
import pandas as pd

bearer_token = "Paste Your API Token"

client = tweepy.Client(
    bearer_token=bearer_token,
    wait_on_rate_limit=True
)

query = "digital marketing -is:retweet lang:en"

data = []

try:
    response = client.search_recent_tweets(
        query=query,
        max_results=20,
        tweet_fields=["created_at", "public_metrics", "author_id"]
    )

    if response.data:
        for tweet in response.data:
            data.append({
                "Tweet": tweet.text,
                "Date": tweet.created_at,
                "Author ID": tweet.author_id,
```

```
                "Likes": tweet.public_metrics["like_count"],
                "Retweets": tweet.public_metrics["retweet_count"],
                "Replies": tweet.public_metrics["reply_count"]
            })
    elif response.errors:
        print("Twitter API returned errors:")
        for error in response.errors:
            print(error)
    else:
        print("No tweets found for the current query.")

except Exception as e:
    print("Twitter API request failed.")
    print(f"Reason: {e}")
    print("Check your internet connection, bearer token, and Twitter API access.")
    raise SystemExit(1)

df = pd.DataFrame(data)

print(df)

df.to_csv("twitter_data.csv", index=False)

print("\nData collected successfully.")
```

Commands to Execute:
pip install tweepy pandas
python Twitter.py

Output (Python):
When the script works successfully, it will print the tweet data in a table format and save it as a CSV file named twitter_data.csv in the same folder. However, in the current run, the script did not fetch data because the Twitter API is Paid and it shows Timed out, so the terminal showed a network/API error instead of tweet output.

Practical No. 4
Data collection using web scraping (BeautifulSoup).

AIM: To collect structured text data from a webpage using Python requests and BeautifulSoup.

Theory: Web scraping means programmatically reading information from webpages. The requests library downloads the webpage response, while BeautifulSoup parses the HTML and provides methods for finding tags and extracting text. Scraping should respect a website's terms, robots rules, rate limits and access restrictions. Only publicly accessible and permitted content should be collected.

Program:
File name: Webscraping.py

```
import requests
from bs4 import BeautifulSoup

url = "https://www.paruluniversity.ac.in/"

response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

print("Title:", soup.title.string if soup.title else "No Title")

h1_tags = soup.find_all("h1")
print("H1 Count:", len(h1_tags))

for h1 in h1_tags:
    print("H1:", h1.get_text(strip=True))

meta_tags = soup.find_all("meta")
print("Meta Tags Count:", len(meta_tags))
```

Commands to Execute:

```
pip install requests beautifulsoup4
python Webscraping.py
```

Output (Python):

Title: Top NAAC A++ Accredited University in Gujarat – Parul University
H1 Count: 1
H1: One Place, Different Paths We have something for everyone!
Meta Tags Count: 18

Practical No. 5
Data preprocessing using Pandas - Cleaning and Standardization.

AIM: To clean and standardize a small social media dataset using Pandas.

Theory: Raw social media data can contain extra spaces, inconsistent capitalization, duplicate rows and empty values. Pandas provides Data Frame operations for cleaning and transforming tabular data. Cleaning before analysis improves the consistency of later tokenization, sentiment analysis and visualization.

Program:
File Name: Clean.py

```
import pandas as pd

data = {
    "user": ["Aman", "Riya", "Aman", "Kabir"],
    "text": [
        " AWS is GREAT! ",
        "Good cloud session",
        " AWS is GREAT! ",
        None
    ]
}

df = pd.DataFrame(data)

df = df.drop_duplicates()
df["text"] = df["text"].fillna("")
df["text"] = df["text"].str.strip().str.lower()

print("Cleaned Data:")
print(df)
```

Commands to Execute:
pip install pandas
python preprocess_Clean.py

Output:

Cleaned Data:

	user	text
0	Aman	aws is great!
1	Riya	good cloud session
3	Kabir	

RESULT: Duplicate, missing and formatting issues were handled using Pandas

Practical No. 6
Data preprocessing using Pandas - Missing Values, Duplicates and Data Types

AIM: To perform common preprocessing operations such as missing-value handling, duplicate removal and data-type conversion using Pandas.

Theory: Preprocessing converts raw data into a consistent form suitable for analysis. Missing values can be removed or replaced depending on the purpose of the dataset. Duplicate records may distort frequency and sentiment results. Pandas also supports conversion of columns to numeric or datetime types, which is useful for filtering and visualization.

Program:
File name: Data.py

```
import pandas as pd


df = pd.DataFrame({
    "post": ["Good product", "Bad service", "Good product", None],
    "likes": ["10", "5", "10", "3"]
})


print(df.info())


df = df.drop_duplicates()
df["post"] = df["post"].fillna("unknown")
df["likes"] = pd.to_numeric(df["likes"], errors="coerce")
df["likes"] = df["likes"].fillna(0)


df.to_csv("processed_social_data.csv", index=False)


print(df)
print("Processed file saved.")
```

Commands to Execute:
pip install pandas
python preprocess_Data.py

Output:

0 Good product 10
1 Bad service 5
3 unknown 3
Processed file saved.

Processed file saved.

post,likes
Good product,10
Bad service,5
unknown,3

RESULT: The dataset was preprocessed using Pandas with appropriate missing-value, duplicate and data-type operations.

Practical No. 7
Tokenization and Stop Word Removal

AIM: To tokenize text into words and remove common stop words using NLTK.

Theory: Tokenization breaks a sentence into smaller units called tokens. Word tokenization is commonly used before text classification and sentiment analysis. Stop words are frequently occurring words such as “the”, “is” and “and” that may contribute less information in some text-mining tasks. Stop-word removal is task-dependent and should not be applied blindly because some common words can carry meaning in sentiment or context.

Program:
File name: nltk_demo.py

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")

text = "This is a very useful cloud computing course."

tokens = word_tokenize(text)
stop_words = set(stopwords.words("english"))

filtered = [
    word for word in tokens
    if word.lower() not in stop_words
]

print("Tokens:", tokens)
print("After stop word removal:", filtered)
```

Commands to Execute:

```
pip install nltk
python nltk_demo.py
```

Output:

```
Tokens: ['This', 'is', 'a', 'very', 'useful', 'cloud', 'computing', 'course', '.']
After stop word removal: ['useful', 'cloud', 'computing', 'course', '.']
```

Practical No. 8
Stemming and Lemmatization

AIM: To apply stemming and lemmatization to text and observe the difference between the two techniques.

Theory: Stemming reduces words by applying simple word-ending rules and may produce a root-like form that is not a dictionary word. Lemmatization uses linguistic information to return a valid base or dictionary form where possible. Stemming is usually faster, while lemmatization can be more linguistically meaningful.

Program:
File name: stem_lemma.py

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize

nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("wordnet")
nltk.download("omw-1.4")

text = "Students are studying studies and running programs."

tokens = word_tokenize(text)

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

stems = [stemmer.stem(word) for word in tokens]
lemmas = [lemmatizer.lemmatize(word) for word in tokens]

print("Original:", tokens)
print("Stemming:", stems)
print("Lemmatization:", lemmas)
```

Commands to Execute:
pip install nltk
python stem_lemma.py

Output:
Original: ['Students', 'are', 'studying', 'studies', 'and', 'running', 'programs', '.']
Stemming: ['student', 'are', 'studi', 'studi', 'and', 'run', 'program', '.']
Lemmatization: ['Students', 'are', 'studying', 'study', 'and', 'running', 'program', '.']

Practical No. 9
Implement Bag of Words Model

AIM: To represent text documents as numerical vectors using the Bag of Words model.

Theory: The Bag of Words model represents a collection of documents using word frequencies. It creates a vocabulary from the training documents and assigns one column to each vocabulary word. The value in a row represents how many times that word occurs in the document. Word order is ignored in the basic model.

Program:
File name: bag_words.py

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "cloud computing is useful",
    "cloud computing is scalable",
    "data analytics is useful"
]

vectorizer = CountVectorizer()
matrix = vectorizer.fit_transform(documents)

print("Vocabulary:")
print(vectorizer.get_feature_names_out())

print("Bag of Words matrix:")
print(matrix.toarray())
```

Commands to Execute:
pip install scikit-learn
python bag_words.py

Output:
Vocabulary:
['analytics' 'cloud' 'computing' 'data' 'is' 'scalable' 'useful']
Bag of Words matrix:
[[0 1 1 0 1 0 1]
 [0 1 1 0 1 1 0]
 [1 0 0 1 1 0 1]]

Practical No. 10
Implement TF-IDF

AIM: To convert text documents into TF-IDF vectors and examine the importance of words across documents.

Theory: TF-IDF stands for Term Frequency–Inverse Document Frequency. Term frequency measures how often a word appears in a document. Inverse document frequency reduces the weight of words that occur in many documents. The combined TF-IDF value highlights words that are relatively important to a document within the collection.

Program:
File name: tfidf_demo.py

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "cloud computing is useful",
    "cloud computing is scalable",
    "data analytics is useful"
]

vectorizer = TfidfVectorizer()
matrix = vectorizer.fit_transform(documents)

print("Features:")
print(vectorizer.get_feature_names_out())

print("TF-IDF matrix:")
print(matrix.toarray())
```

Commands to Execute:
pip install scikit-learn
python tfidf_demo.py

Output:
Features:
['analytics' 'cloud' 'computing' 'data' 'is' 'scalable' 'useful']
TF-IDF matrix:
[[0. 0.52682017 0.52682017 0. 0.40912286 0.
0.52682017]
[0. 0.4804584 0.4804584 0. 0.37311881 0.63174505
0.]
[0.5844829 0. 0. 0.5844829 0.34520502 0.
0.44451431]]

Practical No. 11
Sentiment Analysis Using Text Blob

AIM: To classify sample social media text as positive, negative or neutral using Text Blob polarity.

Theory: Sentiment analysis identifies the emotional orientation expressed in text. Text Blob provides a simple polarity score ranging from -1 to +1. Positive values generally indicate positive sentiment, negative values indicate negative sentiment and values close to zero indicate neutral sentiment. The score is an automated linguistic estimate and should be interpreted according to the text and application.

Program:
File name: text_blobdemo.py

```
from textblob import TextBlob


posts = [
    "The new cloud service is excellent.",
    "The application is slow and frustrating.",
    "The workshop starts at 10 AM."
]


for post in posts:
    polarity = TextBlob(post).sentiment.polarity


    if polarity > 0:
        label = "Positive"
    elif polarity < 0:
        label = "Negative"
    else:
        label = "Neutral"


    print(post)
    print("Polarity:", round(polarity, 2))
    print("Sentiment:", label)
    print()
```

Commands to Execute:
pip install textblob
python text_blobdemo.py

Output:
The new cloud service is excellent.
Polarity: 0.57
Sentiment: Positive

The application is slow and frustrating.
Polarity: -0.35
Sentiment: Negative

The workshop starts at 10 AM.
Polarity: 0.0
Sentiment: Neutral

Practical No. 12
Sentiment Analysis Using NLTK

AIM: To perform sentiment analysis on social media text using NLTK's VADER sentiment analyzer.

Theory: VADER (Valence Aware Dictionary and Sentiment Reasoner) is a lexicon- and rule-based sentiment analyzer included in NLTK. It is designed particularly for short social-media-style text and returns positive, negative, neutral and compound scores. The compound score is a normalized overall sentiment score.

Program:
File name: nltk_demo.py

```
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer


nltk.download("vader_lexicon")


sia = SentimentIntensityAnalyzer()


posts = [
    "I love this new feature!",
    "This update is terrible.",
    "The class starts at 10 AM."
]


for post in posts:
    scores = sia.polarity_scores(post)
    compound = scores["compound"]


    if compound >= 0.05:
        label = "Positive"
    elif compound <= -0.05:
        label = "Negative"
    else:
        label = "Neutral"
```

```
    print(post)
    print(scores)
    print("Sentiment:", label)
    print()
```

Commands to Execute:

```
pip install nltk
python nltk_demo.py
```

Output:

```
I love this new feature!
{'neg': 0.0, 'neu': 0.4, 'pos': 0.6, 'compound': 0.6696}
Sentiment: Positive

This update is terrible.
{'neg': 0.508, 'neu': 0.492, 'pos': 0.0, 'compound': -0.4767}
Sentiment: Negative

The class starts at 10 AM.
{'neg': 0.0, 'neu': 1.0, 'pos': 0.0, 'compound': 0.0}
Sentiment: Neutral
```

Practical No. 13
Build ML Model (Naïve Bayes / Logistic Regression)

AIM: To build a simple text classification model using TF-IDF features and Logistic Regression.

Theory: Machine learning classification learns patterns from labeled examples. In text classification, text is first converted into numerical features such as TF-IDF vectors. Logistic Regression is a supervised classification algorithm that can be used for binary or multiclass classification. The model is trained on labeled examples and then used to predict labels for unseen text.

Program:
File name: ml_sentiment.py

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


texts = [
    "excellent service", "very good product",
    "amazing experience", "happy with the result",
    "bad service", "terrible product",
    "very poor experience", "I dislike this result"
]


labels = [
    "positive", "positive", "positive", "positive",
    "negative", "negative", "negative", "negative"
]


X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, random_state=42
)
```

```
vectorizer = TfidfVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)


model = LogisticRegression()
model.fit(X_train_vec, y_train)


pred = model.predict(X_test_vec)
print("Accuracy:", accuracy_score(y_test, pred))


new_text = ["excellent product and good service"]
new_vec = vectorizer.transform(new_text)
print("Prediction:", model.predict(new_vec)[0])
```

Commands to Execute:

```
pip install scikit-learn
python ml_sentiment.py
```

Output:

```
Accuracy: 0.5
Prediction: positive
```

Practical No. 14
Data Visualization Using Matplotlib / Seaborn

AIM: To visualize social media data using Matplotlib and Seaborn.

Theory: Data visualization converts numerical information into charts that make patterns easier to inspect. Matplotlib provides general-purpose plotting functions, while Seaborn provides higher-level statistical graphics built on Matplotlib. Common social media visualizations include bar charts for engagement, line charts for trends and count plots for category frequencies.

Program:
File name: visualization_demo.py

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

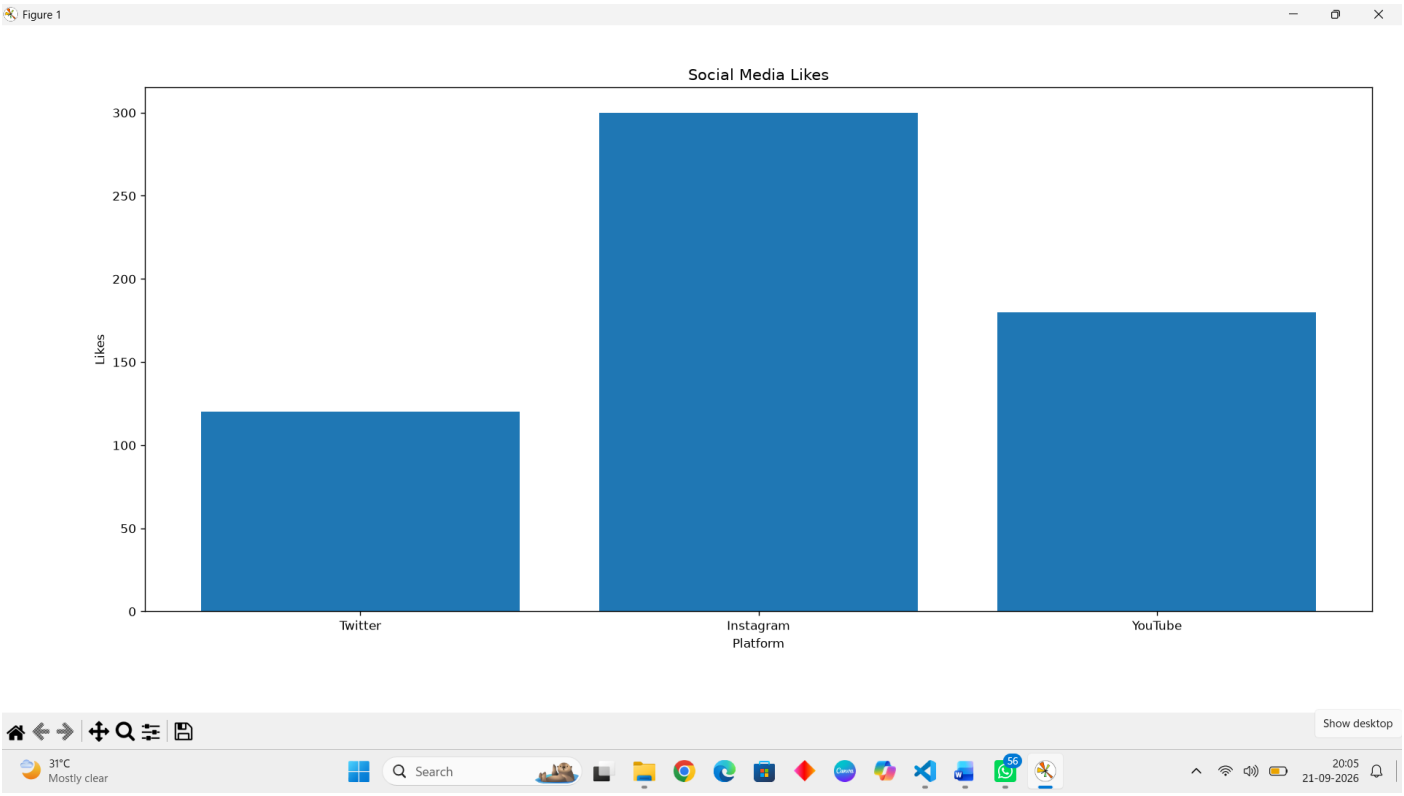
df = pd.DataFrame({
    "platform": ["Twitter", "Instagram", "YouTube", "Twitter", "Instagram"],
    "likes": [120, 250, 180, 90, 300]
})

plt.figure(figsize=(7, 4))
plt.bar(df["platform"], df["likes"])
plt.title("Social Media Likes")
plt.xlabel("Platform")
plt.ylabel("Likes")
plt.tight_layout()
plt.show()

plt.figure(figsize=(7, 4))
sns.countplot(data=df, x="platform")
plt.title("Number of Records by Platform")
plt.tight_layout()
plt.show()
```

Commands to Execute:
pip install pandas matplotlib seaborn
python visualization_demo.py

Output:



A bar chart showing likes by record and a count plot showing the number of records for each platform.

RESULT: Social media data was visualized using Matplotlib and Seaborn.

Practical No. 15
Mini Project: Social Media Sentiment Analysis (End-to-End)

AIM: To build an end-to-end mini project that cleans social media text, performs sentiment analysis, summarizes results and visualizes the sentiment distribution.

Theory: An end-to-end social media sentiment workflow combines data collection, preprocessing, analysis and visualization. A practical pipeline may contain data collection from an API or permitted web source, cleaning, tokenization, sentiment scoring, aggregation and visualization. Keeping each stage separate makes the workflow easier to test and explain.

Program:
File name: mini_project.py

```
import pandas as pd
import nltk
import matplotlib.pyplot as plt
from nltk.sentiment import SentimentIntensityAnalyzer

nltk.download("vader_lexicon")

df = pd.DataFrame({
    "text": [
        "I love this product!",
        "The service is excellent.",
        "This is a terrible experience.",
        "The update is very slow.",
        "The workshop starts tomorrow."
    ]
})

df = df.drop_duplicates()
df["text"] = df["text"].fillna("").str.strip()

sia = SentimentIntensityAnalyzer()
```

```
def get_sentiment(text):
    score = sia.polarity_scores(text)["compound"]
    if score >= 0.05:
        return "Positive"
    if score <= -0.05:
        return "Negative"
    return "Neutral"


df["sentiment"] = df["text"].apply(get_sentiment)

print(df)
print("\nSentiment counts:")
print(df["sentiment"].value_counts())

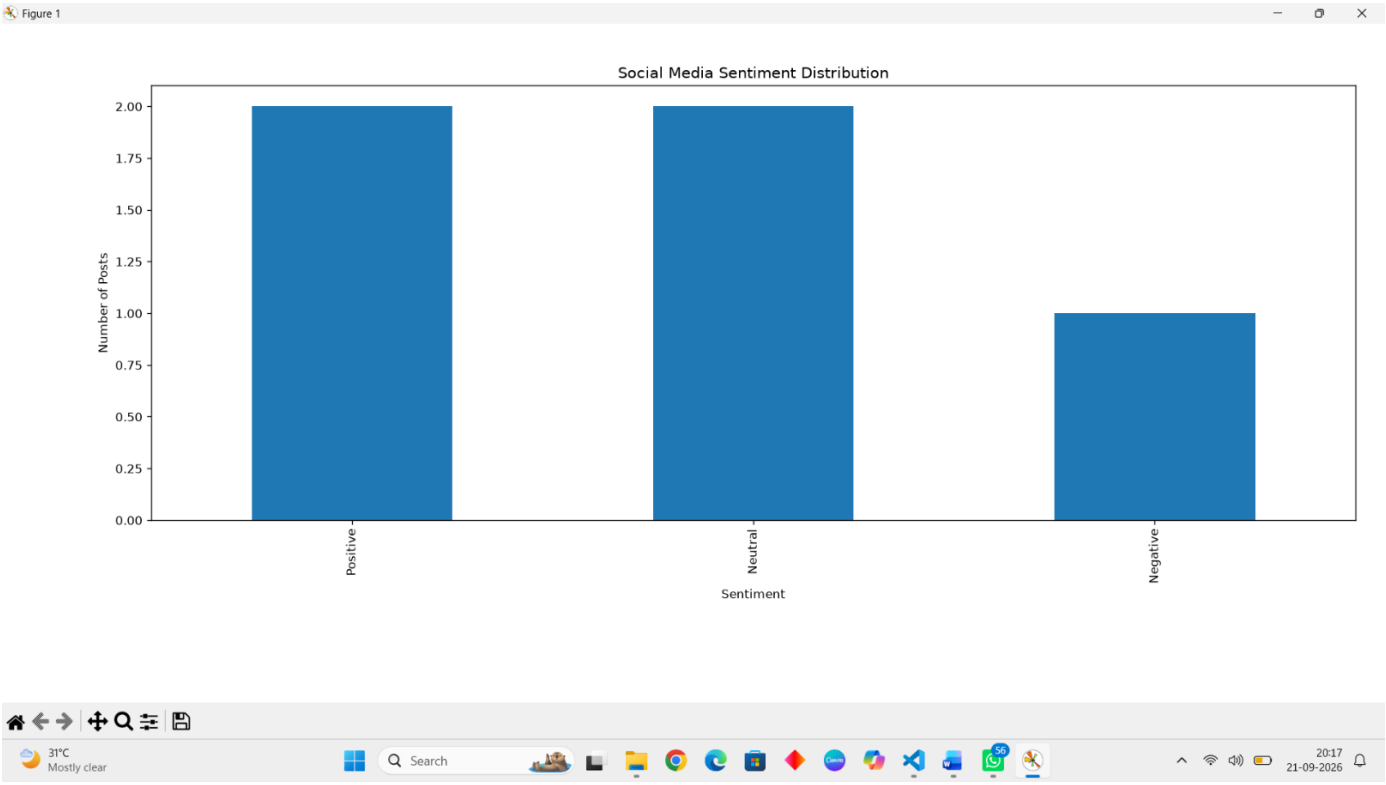
df.to_csv("sentiment_results.csv", index=False)

plt.figure(figsize=(7, 4))
df["sentiment"].value_counts().plot(kind="bar")
plt.title("Social Media Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Number of Posts")
plt.tight_layout()
plt.show()
```

Commands to Execute:

```
pip install pandas nltk matplotlib
python mini_project.py
```

Output:



	text	sentiment
0	I love this product!	Positive
1	The service is excellent.	Positive
2	This is a terrible experience.	Negative
3	The update is very slow.	Neutral
4	The workshop starts tomorrow.	Neutral

Sentiment counts:

sentiment	
Positive	2

RESULT: An end-to-end social media sentiment analysis pipeline was implemented successfully.

Parul® University
Vadodara, Gujarat

NAAC RA A++



https://paruluniversity.ac.in/